Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3776585

PDF Download
3776585.pdf
05 February 2026
Total Citations: 0
Total Downloads: 182

Published: 13 December 2025
Online AM: 11 November 2025
Accepted: 26 October 2025
Revised: 22 October 2025
Received: 06 November 2024

Citation in BibTeX format

RESEARCH-ARTICLE

SoCurity: Enhancing SoC Security with Anomalous Activity Detection and Localization

NAORIN HOSSAIN, International Business Machines, Armonk, NY, United States

ALPER BUYUKTOSUNOGLU, International Business Machines, Armonk, NY, United States

JOHN DAVID WELLMAN, International Business Machines, Armonk, NY, United States

PRADIP BOSE, International Business Machines, Armonk, NY, United States

MARGARET MARTONOSI, Princeton University, Princeton, NJ, United States

Open Access Support provided by:

International Business Machines

Princeton University

ACM Transactions on Architecture and Code Optimization, Volume 22, Issue 4 (December 2025)
https://doi.org/10.1145/3776585
EISSN: 1544-3973

SoCurity: Enhancing SoC Security with Anomalous Activity Detection and Localization

NAORIN HOSSAIN, Research, IBM, Yorktown Heights, United States
ALPER BUYUKTOSUNOGLU, Research, IBM, Yorktown Heights, United States
JOHN-DAVID WELLMAN, Research, IBM, Yorktown Heights, United States
PRADIP BOSE, Research, IBM, Yorktown Heights, United States
MARGARET MARTONOSI, Computer Science, Princeton University, Princeton, United States

As systems-on-a-chip (SoCs) that power our devices become increasingly heterogeneous to meet low-power, high-performance computational requirements, they also become more vulnerable to the possible impacts of an on-chip resource availability attack. In this article, we present SoCurity, the first network-on-chip (NoC) counter-based hardware monitoring approach for enhancing heterogeneous SoC security. With this monitoring approach, we develop a fast, lightweight anomalous activity detection and localization system at the hardware level. The presented detection system uses semi-supervised machine learning models, and requires no prior attack knowledge for detecting anomalous activity. This design choice provides protection against existing and novel future attacks impacting on-chip resource availability. Our localization engine leverages the inherent interpretability of SoCurity's NoC counters to locate detected anomalous activity in the SoC for focused recovery. We demonstrate our techniques with a case study on a real SoC implementation for a connected autonomous vehicle system and find up to 96% detection accuracy and fast detection (~30μs for prediction on an ASIC) in online experiments. Furthermore, we show that the flagged anomalous activity can be reliably localized for up to 99% of detected anomalous activity in our experiments.

CCS Concepts: • **Security and privacy** → *Intrusion/anomaly detection and malware mitigation*; *Embedded systems security*; • **Computer systems organization** → **System on a chip;**

Additional Key Words and Phrases: System-on-a-chip, anomaly detection, denial-of-service attack, network-on-chip, hardware performance counters

ACM Reference Format:
Naorin Hossain, Alper Buyuktosunoglu, John-David Wellman, Pradip Bose, and Margaret Martonosi. 2025. SoCurity: Enhancing SoC Security with Anomalous Activity Detection and Localization. *ACM Trans. Arch. Code Optim.* 22, 4, Article 166 (December 2025), 26 pages. https://doi.org/10.1145/3776585

Authors' Contact Information: Naorin Hossain (corresponding author), Research, IBM, Yorktown Heights, New York, United States; e-mail: naorin.hossain@ibm.com; Alper Buyuktosunoglu, Research, IBM, Yorktown Heights, New York, United States; e-mail: alperb@us.ibm.com; John-David Wellman, Research, IBM, Yorktown Heights, New York, United States; e-mail: wellman@us.ibm.com; Pradip Bose, Research, IBM, Yorktown Heights, New York, United States; e-mail: pbose@us.ibm.com; Margaret Martonosi, Computer Science, Princeton University, Princeton, New Jersey, United States; e-mail: mrm@princeton.edu.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 1544-3973/2025/12-ART166
https://doi.org/10.1145/3776585

1 Introduction

Systems-on-a-chip (**SoCs**) consolidate various processing components on a single die to offer high-performance processing ability, while maintaining low area and power overheads. They are increasingly being adopted to power digital systems, including personal computers, servers, and embedded devices, such as smartphones, autonomous vehicles, medical devices, data centers, and the internet-of-things [9, 61]. Heterogeneous SoCs are designed to the target device's computational needs by bringing together specialized processing units tailored to accelerate the device's tasks, all on a single die. SoC components can include general purpose processing cores, hardware accelerators, memory interfaces, and I/O interfaces. Some of the SoC components may also be black-box third-party **intellectual property** (**IP**) units. State-of-the-art SoCs use **network-on-chip** (**NoC**) interconnects for efficient communications between these SoC components [6, 15].

Although heterogeneous SoCs make notable improvements in power and performance, they also introduce more severe security challenges than prior system designs. Prior homogeneous system designs could be protected more easily with well-established and consistent security approaches for processing cores throughout the system. In contrast, the diverse set of components in heterogeneous systems invites the possibility of vulnerabilities to be introduced and exploited in a variety of ways throughout the system. Furthermore, integration of components from third-party vendors presents the added risk of unknown vulnerabilities introduced by internal component designs. It is thus a substantially greater challenge for systems designers to ensure security throughout a heterogeneous SoC [45].

With the increased attack space from heterogeneous components, sophisticated attacks can be designed to negate power and performance gains achieved by specialized SoC designs by targeting and taking down SoC components, rendering them unavailable. Figure 1 illustrates one avenue for bringing down SoC units: malware that launches a **denial-of-service** (**DoS**) attack [15]. Such malware can flood the NoC interface with packets targeting specific SoC resources. The resulting NoC congestion can prevent necessary tasks from reaching victim components, and can slow packet transmissions for other units. Such attacks can similarly be launched by other mechanisms, e.g., hardware trojans that have been integrated into malicious third-party IP [8, 14, 47, 56, 63]. To preserve the power and performance gains of SoCs, it is of utmost importance to design built-in security measures to protect SoCs from such threats and limit performance impact caused by them. Our goal is to present a widely applicable SoC design enhancement that can provide such protection against resource availability attacks through detection and localization methodology that can ease and enable focused threat mitigation.

Although prior works used hardware performance counters for malware detection in traditional CPU architectures [18, 37, 51, 52], they are insufficient for SoCs as they may not be able to monitor the different processing units, memory, and I/O interfaces on SoCs that can be targeted by malware. Furthermore, black-box SoC components may not expose desired hardware performance counters to be used in such an approach. SoC-specific works have not provided reliable or broadly applicable threat detection solutions for heterogeneous SoCs (Section 7). With rapidly changing SoC designs, it is critical for these needs to be met. Our design solution proposes methodology that is better suited for built-in security on heterogeneous SoCs.

The SoCurity Approach: This article proposes SoCurity, a design method for enhancing security in heterogeneous SoCs at the NoC level. SoCurity presents the first use of *NoC-based hardware counters* to monitor ongoing SoC activity. NoC-level monitoring ensures there is no reliance on the internal design of individual SoC components. NoC-based counters can thus provide a foundation for hardware-level security measures to be incorporated into a broad scope of SoC designs. It is also a particularly powerful solution in the face of rapidly changing SoC designs and runtime reconfigurable designs.

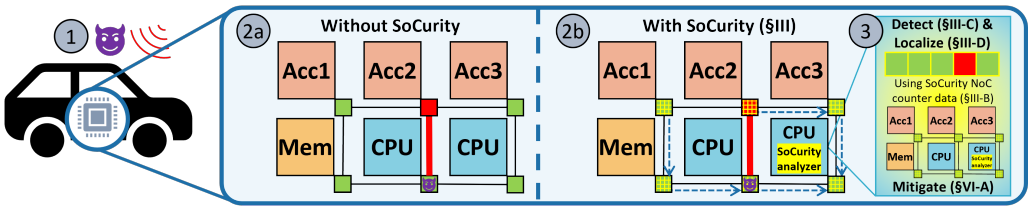


Fig. 1. Example attack on SoC (related case study in Section 4): ① Malware introduced from network data transfer. ②a Malware launches DoS attack on Acc2. Red router and NoC link indicate increased NoC traffic. Green routers experience regular traffic. In contrast, ②b and ③ show how SoCurity helps. ②b SoC enhanced with SoCurity in its base configuration, where anomaly detection and localization (SoCurity analyzer) is implemented on a core in the SoC. NoC counters are added to routers (yellow boxes). NoC counter data is collected and sent to SoCurity analyzer on an isolated NoC plane (dashed arrows). ③ Anomalous NoC data (red) is detected in counters by SoCurity analyzer. The impacted SoC component is derived from counters by the SoCurity analyzer's localization engine. Location information is used to diagnose and mitigate threat.

Built on top of SoCurity's NoC-based SoC monitoring, we developed a fast, lightweight anomalous activity detection and localization system, called the SoCurity analyzer, that can capture hardware availability attacks and identify where anomalous activity is occurring in the SoC. Here, lightweight indicates low algorithmic complexity with compact processing and memory requirements resulting in swift execution, even on small-scale embedded systems. To account for the growing attack space as SoC designs rapidly change and become more heterogeneous, no prior knowledge or characterization of attacks is required with our approach, enabling detection of existing attacks and novel future availability attacks. To this end, we used *semi-supervised machine learning (ML) models* that train only on benign SoC behavior captured by the NoC counter data. As described in Section 3.3.3, we tested several semi-supervised models for effectiveness in our study. After training, the model is deployed on the SoC to identify and flag counter data that deviates from training data as *anomalous* activity, e.g., excessive NoC traffic from a DoS attack on an SoC component (Figure 1).

When anomalous activity is detected, the counter data is then analyzed with a rule-based explainable ML algorithm that is used to hone in on the source and victim tiles in which the anomalous activity is occurring by finding the counters that most deviate from benign training data. This localization process is particularly useful for mitigation systems to take action on focused parts of the SoC accordingly, e.g., by shutting down or restarting only the impacted components such that a full system reboot is not needed (though note that mitigation is outside the scope of this work). Furthermore, during the SoC's operation, automated periodic updates can be made to the trained detection model and localization algorithm by incorporating new NoC counter data that is determined to be benign. Anomaly detection and localization with the SoCurity analyzer thus presents an opportunity for systems to be designed with integrated security, providing a means for end-to-end anomaly handling at the hardware level.

This article presents the following contributions:

- SoCurity, a novel SoC monitoring approach for enhancing security using NoC-based hardware counters that is easily integrated into SoC designs.
- A fast, lightweight, automated anomaly detection system targeting unwarranted on-chip resource usage for SoCs.
- Our system needs no attack knowledge for detection; developed with adaptive detection based on semi-supervised ML models, it enables novel future attack detection.

— A localization algorithm that leverages data provided by NoC counters and explainable ML techniques. Our study provides key insights on interpreting NoC counter data for developing such an algorithm for focused mitigation.
— We demonstrate these techniques in a case study for a real connected autonomous vehicle SoC design in high variability settings. Our experiments leverage two SoC configurations to showcase SoCurity's detection and localization capability in single and multicore settings. Our results show highly accurate DoS detection (up to 96%), fast prediction times (~30µs on an ASIC) in online tests, and highly reliable localization (up to 99%).

2 Background and Motivation

In this section, we discuss some of the challenges that come with the ever-increasing heterogeneity of SoCs powering modern systems. This motivates the presented SoCurity work.

2.1 Designing SoCs

With the end of Moore's Law and Dennard scaling, specialized application-specific hardware designs have been required to continue achieving performance gains [9]. Heterogeneous SoCs maximize energy efficiency by curating processing units on chips to provide optimal performance for target devices. Heterogeneous SoC components can include a variety of processing units like CPUs, GPUs, other hardware accelerators, and interface tiles for memory and I/O. These components may include third-party IP that may be black-box tiles or runtime reconfigurable components [25, 49, 59, 62, 68].

Given the wide range of designs that are rapidly being developed for different devices, it can be challenging to ensure that SoC correctness and security keeps up and is maintained across these fast-changing systems. The complexity of designing, validating, and programming heterogeneous SoCs has led to recent efforts for an open-source hardware design approach with the goal of integrating SoC components that are *independently* designed and can be reused in new SoCs [44], including black-box third-party IP. The potential inability to observe hardware activity in regions of the SoC composed of third-party IP requires new methods to monitor SoC activity that can still provide detection of security breaches.

One such approach to monitoring SoC activity, regardless of its composition, is through the NoC interconnect. The NoC is composed of routing logic at each component, and one or more planes which transport packets between components. Multiple NoC planes can exist to facilitate packet transfers [69]. A NoC can be implemented using any of a variety of topologies, e.g., a 2D mesh or torus [46]. In this work, we take advantage of the NoC interconnect in the SoC, which provides a neutral ground for monitoring SoC activity regardless of its composition.

2.2 Runtime Availability of SoC Components

Heterogeneous SoC performance gains rely on parallelizing tasks across on-chip processing units and accelerating key computations with specialized hardware. These gains begin to falter when either of these capabilities are compromised, and system safety may be compromised as a result. It is thus critical to identify availability attacks, which target availability of any SoC components, in order to maintain the functionality and benefits of heterogeneous designs. The SoCurity work presented in this article focuses on the most common availability attack, DoS attacks [39], though other attack protection is also possible and is discussed further in Section 6.

Historically, DoS attacks have been used to flood networks, bringing them down. The limited security of embedded IoT devices presented an easy target for creating botnets to perform distributed DoS attacks. For example, from 2014–2016, the Mirai malware was used to infect hundreds of thousands of IoT devices to create botnets launching distributed DoS attacks against several high-profile targets (e.g., websites and university networks) [3]. SoCs present a new landscape for DoS

attacks to target internal resources. Malware and hardware trojans can be leveraged to launch DoS attacks on SoC components [15]. They can cripple the system with simple memory requests that congest the NoC, preventing other processes from accessing memory on time. They can also flood the NoC with accelerator tasks, causing slowdown in key computations requiring specialized hardware for fast execution, making them particularly harmful in embedded devices with real-world impacts. For example, a DoS attack can hang up SoC components in a smart medical sensor device and prevent a patient from receiving needed care. Likewise, DoS attacks can slow down data processing and cause incorrect decisions in autonomous vehicles that can lead to longer travel times or even accidents.

Hardware-level solutions to protect against availability attacks like DoS on SoCs can enable fast detection and recovery. *Since these attacks are designed to cause disruptions to standard operation, the ongoing SoC activity at the hardware level is distinguishable during an attack.* This work thus leverages hardware activity monitoring for SoCs as the means for developing an anomalous activity detection and localization system capable of flagging and finding DoS attacks in the system. Given the changing landscape in SoC designs, such a solution requires adaptability to different heterogeneous SoCs. In this article, we present a design approach that can be easily applied to SoCs with any composition of processing units.

3 SoCurity: NoC Counter-Based SoC Security

SoCurity enhances security measures on a heterogeneous SoC using NoC-based hardware counters. Using SoCurity, this section develops the SoCurity analyzer, an example of an anomalous activity detection and localization system that can be implemented with this SoC design approach. Figure 2 provides an overview of SoCurity, including the NoC counter approach to monitoring SoC activity and the SoCurity analyzer's anomalous activity detection and localization system.

3.1 Threat Model

The work in this article presents hardware-level security designs for maintaining the performance gains made possible by heterogeneity in SoCs. In particular, this work seeks to protect the SoC against attacks that disturb the regular operation and accessibility of components on the SoC. SoCurity is thus built to protect against hardware-level threats like hardware trojans that are incorporated at design time into third-party IP tiles, as well as software-level threats like malware injected at runtime through connected devices or the cloud, respectively. Though the techniques presented in this work can detect anomalous hardware activity resulting from either of these attack types, our case study (Section 4) focuses on malware injections that launch DoS attacks, the most common hardware availability attack and a concerning vulnerability for SoCs with NoCs [15, 39]. As discussed in Section 6 though, SoCurity can enable detection of other threats as well.

In our threat model, SoC components (i.e., tiles) are considered untrusted as they may be from third-parties, but the NoC is a trusted component that should not be provided by third-parties. The SoC's software stack including the OS is considered to be untrusted. SoCurity is built as a separate monitoring unit that is not part of the SoC's untrusted software stack. We conservatively assume the attacker is aware of accelerators on the SoC and is able to send tasks to them, through means such as reverse-engineering the SoC composition and manipulating access permissions. No particular SoC configuration is targeted by attackers, rather on-chip accelerators and availability of other resources is the main target so that the attacker can prevent continued regular operation of the system. These are broad attacks that may impact any SoC configuration for any device.

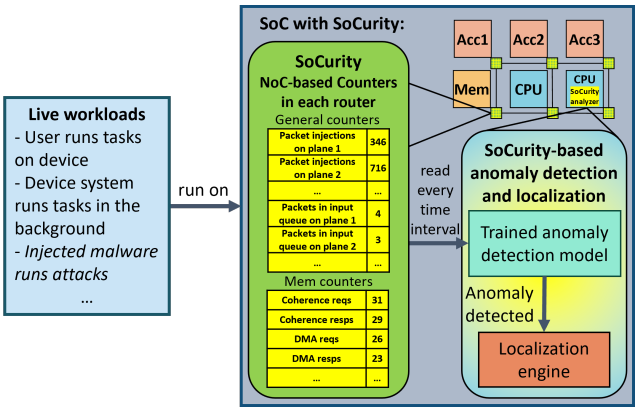


Fig. 2. SoCurity introduces counters to NoC routers for each tile in an SoC to monitor ongoing interactions between tiles (example counters shown in yellow). These counters enable a constantly active anomaly detection and localization system with the SoCurity analyzer while live workloads run on the SoC.

3.2 NoC-Based SoC Hardware Counters

In a heterogeneous SoC, monitoring hardware activity across various on-chip processing units and black-box third-party IP can be achieved by either implementing a standard interface in each component or by monitoring at the NoC level. SoCurity takes the latter approach, as the NoC interconnect granularity provides a reliable medium for monitoring component usage and communication throughout the SoC without imposing additional requirements on component designs. This approach is ideal for hardware designs like those supported by the ESP project [44], a plug-and-play platform that integrates independently designed accelerators into SoCs within a 2D mesh NoC. SoCurity is a natural fit for SoC design platforms like ESP as it provides an open-source customizable NoC.

NoC counters can capture a holistic view of ongoing SoC activity by monitoring (1) NoC packets injected by each tile, (2) NoC congestion at each tile, (3) memory tile requests and responses, and (4) accelerator usage. As Section 4 describes, hundreds of counters may be present in the SoC to monitor these events. However, depending on the system, a small subset of them are likely to be sufficient for active threat monitoring by the SoCurity analyzer, reducing counter communication load.

Counters can be implemented as protected registers at the router for each tile (such as the counter examples shown in Figure 2). NoC packet counters can increment at each source and destination router for each packet. NoC congestion counters can track packet input queue size at each router. For memory interface tiles, some counters can track coherence and **direct memory access** (**DMA**) requests by incrementing (on arrival) for packets from cores and accelerators, respectively. Routers for accelerators can count usage cycles by recording cycles between a packet arrival from a core and a response packet departure for that core. Our proposed counters are generic and applicable for tiles in any SoC design and NoC topology. To our knowledge, no prior work has leveraged NoC-based counters for holistic anomaly detection in heterogeneous SoCs.

3.3 Anomaly Detection Methods

The proposed NoC counter-based monitoring approach generates a stream of data that must be analyzed in order to detect *anomalous activity*. Anomalous activity is a deviation from a device's regular system activity that may result from malicious actors or problems with regular hardware

functions. Detecting anomalies is common practice in defense systems. This section presents the detection system we developed for the SoCurity analyzer to perform this data analysis and flag anomalous activity occurring in an SoC.

3.3.1 Striking the Right Balance in Detection Complexity. The goal for our system is a lightweight algorithm that quickly flags uncharacteristic hardware behavior without searching behaviors specific to known attacks such that novel attacks may be detected as well. Although a simple approach is desirable, our system should be more robust than naive solutions like monitoring latency changes or threshold-based algorithms, which we have found to be inaccurate in complex SoC usage scenarios, resulting in high false positives and false negatives. Latency increases can result from many implementation-specific reasons, such as swapping pages in and out of memory. Threshold-based algorithms may lead to false detection results as peaks in some hardware activity are not always indicative of anomalous behavior, while activity falling under defined caps may be anomalous. With more robust methods, SoC activity can be analyzed *in the context of all other ongoing activity in the SoC*, improving overall detection accuracy. On the other hand, complex solutions using **deep neural networks** (**DNNs**) may provide very accurate detection, but require immense memory, storage, and processing resources to do so [41]. Such large models are not ideal for fast detection in devices with limited resources. For broader applicability, we explore more accessible ML-based anomaly detection models that can provide both accurate and fast detection with lighter computational requirements.

3.3.2 ML Models for Anomaly Detection. ML-based anomaly detection models can be trained in different ways. *Supervised* models are trained using fully labeled datasets with regular and anomalous data. Some datasets may not be labeled though, as accurate labeling often requires tedious manual effort that can be infeasible with large datasets. For these, *unsupervised* algorithms may be used which do not require training on pre-defined labels. Instead, regular and anomalous data are analyzed assuming the majority of data is not anomalous. Outliers are picked out and labeled as anomalies. However, this assumption of anomalous data being in the minority is not sufficient in ensuring system security and may easily be thwarted by a malicious actor that overwhelms the system with DoS attacks or similar, creating more anomalous data than benign.

Less commonly studied is use of *semi-supervised* models which can be trained with partially labeled data. For our SoCurity analyzer, we studied a subset of semi-supervised ML models that are specific to anomaly detection. They are binary classification models that are trained solely with benign data; anomalous data need not be included in the training set. This eases the training data collection process, as representative benign datasets are much easier to collect than anomalous datasets that capture all possible attack examples. Training on benign data also avoids the pitfall of unsupervised detection models that may allow DoS attacks to evade them by overwhelming the system with anomalous data. Collected training data is used to define bounded regions of benign activity. Samples that fall outside of these regions get labeled *anomalous*. With no anomalous training data, these models are capable of capturing both existing and novel future attacks, a particularly attractive quality that inspired their use here.

3.3.3 Semi-Supervised Models for SoCurity. We considered four semi-supervised anomaly detection models for use in our SoCurity analyzer's detection system. They are based on common supervised models, but designed for one-class classification, training only on a set of benign data. These models are well-studied anomaly detection algorithms and their simplicity, speed, accuracy, and general applicability best fit our goals [1, 13]. For our case study (Section 4), we used available model implementations from the Python scikit-learn library [53], with the exception of the **one-**

class nearest neighbors (**OCNN**) model which is not available. We thus wrote our own OCNN implementation based on the **K-Nearest Neighbors** (**KNN**) module from the scikit-learn library.

One-Class Nearest Neighbors: Nearest neighbors algorithms like KNN use distance between samples for prediction [24]. Training data is stored as raw feature vectors. Distances from test data to each training datum are calculated, with a majority vote among the k nearest neighbor labels used to classify a test sample. In the "one-class" anomaly detection variant, OCNN, a distance boundary determines whether a sample is close enough to training data to label it benign [32].

One-Class Support Vector Machines: Support vector machines (**SVM**) distinguish classes by projecting data to a higher dimension where samples can be separated by a hyperplane [16]. The nearest training points to the hyperplane are the support vectors, and the optimal hyperplane maximizes distance to them. In **one-class SVM** (**OCSVM**), samples are lifted to a higher dimension but the optimal hyperplane maximizes distance between support vectors in the benign training set and the origin [57]. The model learns a frontier where training samples are densely populated. Anomalies fall outside this region when raised to the higher dimensional space.

Isolation Forest: Isolation forest (**iForest**) is a one-class, anomaly detection variant of the supervised random forest model [40]. Random forest is an ensemble model composed of several decision trees that break up a classification problem [11]. iForest is similarly composed of **isolation trees** (**iTrees**) that split on random features in the training set until all data resides at a leaf node. iForest is based on the notion that anomalies are distinct enough from regular data that they can be quickly and easily sifted out. Anomalies thus have a shorter path down an iTree than a regular sample. Path length scores across all iTrees are used to flag anomalous samples.

Local Outlier Factor: Local outlier factor (**LOF**) is a distance-based algorithm similar to kNN, but also accounts for local density of samples [12]. As regular samples are likely to be densely packed in a space, anomalies would thus be farther away from these regions, in sparser areas of the space. LOF scores indicate a sample is benign when it falls within a dense cluster of its neighbors.

3.4 SoCurity Counters for Explainable Anomaly Localization

A key benefit of using SoCurity's NoC counters is its capability to aid in diagnosing anomalous activity by providing data on the location and nature of the activity within the SoC, i.e., facilitating the *localization* of anomalous events. SoCurity's counters, distributed throughout the SoC, continuously collect data indicative of ongoing activity at each component (see Section 3.2). When counter data is flagged as anomalous, the SoCurity analyzer's anomaly detection engine passes it to the localization engine, which analyzes the data to pinpoint the anomalous activity in the SoC, enabling mitigation efforts to focus on recovery for only impacted components, greatly reducing performance impact on the system during recovery.

To localize with NoC counter data, explainable ML techniques can be leveraged, a burgeoning field aimed at providing *interpretability* to ML model outcomes [4]. Some inherently explainable models, like decision tree-based ones (e.g., random forests and iForests), examine decision nodes to understand classification factors. In contrast, models like SVM tend to be more black-box, but tools such as **Local Interpretable Model-Agnostic Explanations** (**LIME**) [54] and SHAP (SHapley Additive exPlanations) [42] offer model-agnostic approaches. These tools use feature importance to understand classification predictions. Similar to decision tree node analysis, another model agnostic approach crafts logical "rules," presented as IF-THEN statements, based on key feature values. These rules provide interpretability to predictions by drawing boundaries around feature values such that data can be categorized into groups that are likely to have similar outcome explanations.

For broad applicability, our anomaly localization approach needs to be fast, lightweight, and model-agnostic, such that the interpretability of the detection engine's ML model is not a factor. Thus for our purposes, tools like LIME and SHAP are not good fits as they can have long runtimes.

Instead, a rule-based algorithm can quickly process NoC counter data to identify where anomalous activity is occurring.

Inspired by prior works [7, 48], we devised a localization approach using rules extracted from benign training data. Regions of benign feature values in the data can be used to define rules beyond which anomalous counter data may lay. Anomalous activity flagged by the detection engine can be checked against these rules to determine which counters contributed to the anomalous classification. As the NoC counters correspond to activity locations in the SoC, pinpointing the anomalous counter values enables *interpretation* of where anomalous activity is occurring in the SoC. For example, increased accelerator and direct memory accesses can indicate an anomalous process making unnecessary task requests to accelerators. Likewise, elevated memory accesses by one or more CPUs may indicate an anomalous process that is continuously making unnecessary memory accesses. In Section 5.5, Figure 10 shows the interpretations for the localization algorithm derived in our case study. Such anomalous NoC counter activity may result from a DoS attack, with counters indicating victim SoC components and the source CPU tile(s) from which the attack is being launched. With SoCurity's support, the detection and localization engines can aid a trusted supervisor or the OS to quickly locate and mitigate the anomalous activity.

3.5 Full SoCurity Anomaly Detection and Localization System

3.5.1 Model Training. Effective anomaly detection with a semi-supervised approach relies on strong training data that accurately portrays regular system behavior. Collecting representative datasets of typical hardware activity in a device is feasible since SoCs are designed for device-specific uses. The detection model is statically trained with counter data collected during workloads that are representative of typical device applications, including the range of possible software or external inputs (e.g., typical sensor inputs for a device with sensors), in a secure, offline setting. This training data is also used to extract the localization rules for the NoC counters (Section 3.4). Once deployed, the trained model and localization rules can be updated periodically with data that is collected during the device's use and found to be benign. These future updates ensure the model remains robust to regular usage.

3.5.2 Secure Implementation. When deployed on an SoC, the base configuration for our proposed SoCurity analyzer is to execute it on an isolated core at the highest privilege level, independent of the software stack, protecting it against malware and attackers. Alternatively, the detection system can be implemented and executed as a standalone hardware unit, enhancing its security from intruders. For either approach, the NoC counters should only be accessible to the SoCurity analyzer through a dedicated NoC plane that is inaccessible to other SoC components by encoding access restrictions in the NoC routing logic. The NoC is thus a trusted part of the SoC and should not be provided by third-parties. Memory for the detection model and counter data should also be isolated in a trusted, secure space that only the SoCurity analyzer can access and do so without interrupting ongoing program executions.

3.5.3 End-to-End Attack Detection and Localization Process. The full anomalous activity defense process works as follows.

(1) NoC counter data is collected while running representative workloads on the SoC in a secure, offline setting.
(2) The anomaly detection model is trained with collected data.
(3) Localization rules are extracted from the training data.
(4) Trained models are deployed on the SoC for runtime detection and localization.
(5) NoC counters are periodically input to the detection model. The period is based on counter collection and detection time.

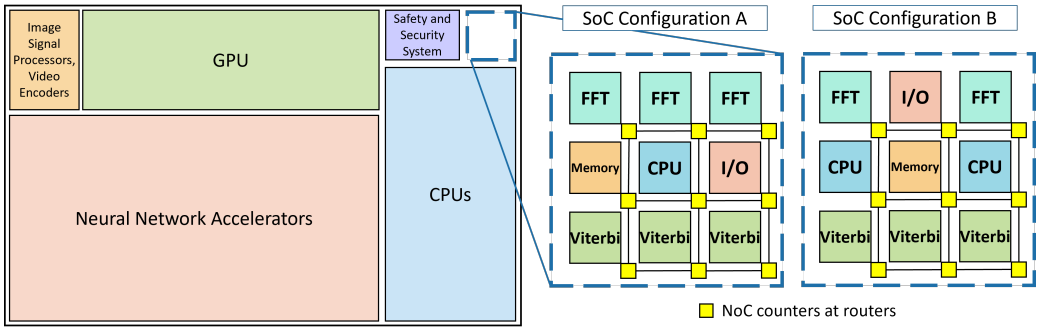


Fig. 3. Overview of the implemented SoC subsystem layouts for each configuration. The SoCs are shown within a larger CAV compute model (based on the Tesla Full Self-Driving chip [6]). They consist of RISC-V Ariane cores, memory tiles, I/O tiles, FFT accelerators, and Viterbi decoder accelerators. SoC A has a single RISC-V CPU, while SoC B has two.

(6) The detection model predicts if the counter data is anomalous.
(7) Anomalous data is provided to the localization engine.
(8) The localization engine runs the anomalous counter data through the localization rules to locate the victim and source SoC units where the anomalous activity is occurring.
(9) The localization engine provides anomaly location data to a trusted supervisor or OS for focused mitigation. Alternatively, the localization engine may be unable to determine an anomalous location, in which case follow-up analysis on more counter data is needed.
(10) Data labeled benign by the SoCurity analyzer is used to update the detection and localization models over time.

4 Case Study: Connected Autonomous Vehicles

To demonstrate SoCurity monitoring, detection, and localization, we performed a case study on real SoC implementations of **connected autonomous vehicle** (**CAV**) systems. CAVs are being designed to improve reliability of navigational decisions through "swarm" communications with nearby vehicles, smart infrastructure, and the cloud for sharing knowledge on surroundings [66]. CAV SoCs are complex and subjected to high input variability based on real-world conditions.

4.1 CAV SoC Implementations

We implemented two SoC prototype configurations for the swarm communication component of a full CAV SoC for this case study (Figure 3). Both SoCs were emulated on a Xilinx Virtex UltraScale+ VCU118 FPGA running at 78MHz using the ESP platform [44]. Each SoC has a 3-by-3 tiled layout. SoC A has one 64-bit RISC-V Ariane core [70], a memory tile, an I/O tile, three FFT accelerators, and three Viterbi decoder accelerators. In contrast, SoC B includes one additional RISC-V core that replaces one of the FFT accelerators. The RISC-V core has a 16KB L1 I-cache and a 32KB L1 D-cache. In SoC B, the CPU tiles also have a 32KB private L2 cache. The memory tile has a 256KB LLC. Tiles are connected by a 2D mesh NoC with six planes for specific data packets [44].

4.1.1 A Representative CAV Workload. For our experiments, we ran a representative workload on the SoC implementation (Figure 4) that addresses characteristics of a real-world CAV application, with a focus on swarm communication. A generated trace of upcoming obstacle positions drives the workload. The workload loops on the following tasks during each *time step*:

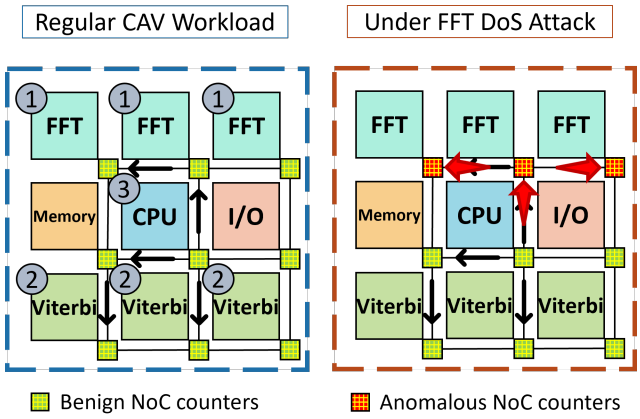


Fig. 4. Example SoC A activity from CAV workload tasks described in Section 4.1.1 are shown on the left SoC. Tiles are numbered 1-3 with tasks from the workload. Benign network traffic is depicted with black arrows. The SoC on the right shows additional traffic from an FFT DoS attack (red arrows).

(1) Calculate distance to surrounding objects using radar readings. This involves FFT calculations performed by the FFT accelerators [28]. FFT task counts per time step depend on the number of supported CAV radar sensings.
(2) Messages are received from other vehicles, infrastructure, or the cloud (i.e., swarm communications). We assume a Viterbi encoding scheme for messages and use Viterbi decoding accelerators to process them [67]. At least one received message is decoded in each time step, with up to two additional messages received and decoded for additional CAV traffic. Four message sizes are used for variable message types across different CAV and infrastructure vendors. Section 5.2 includes experimental results from further increasing variability of incoming message traffic to stress test SoCurity detection.
(3) Combine learned knowledge to plan and execute actions. This step is driven by information from steps 1 and 2.

4.1.2 Implementing NoC-Based Counters. For this study, we deployed NoC-based hardware counters (Section 3.2) in our SoC implementations using the lightweight hardware monitoring system from the Cohmeleon work for ESP, which has negligible area cost (<0.1%) [72]. These monitors are implemented as memory-mapped registers in each SoC tile. Mapped to a contiguous region of I/O address space, they can be accessed in software with standard Linux system calls. We monitor 355 total NoC counters in SoC A and 395 total NoC counters in SoC B. These counters are:

— **General:** NoC packets, backpressure cycles by tile/plane
— **Memory:** off-chip memory accesses, coherence requests/responses, DMA requests/responses, LLC hits/misses
— **Accelerator:** total/memory/TLB cycles, invocations[1]

Note that these counters are based on NoC activity and are distinct from CPU performance counters. Memory counters are for memory tile activity, not CPU memory instructions.

[1]The accelerator invocation counter was added to SoC B only. It strongly correlates with the other accelerator counters.

4.2 DoS Attack Targets

We ran four DoS attack variants on the SoCs by flooding the NoC with packets for victim tiles. These attacks are realistic examples of how malware may abuse access to on-chip hardware resources as they are implemented by making continuous accesses to these resources, just as a network DoS attack may be implemented to take down a node in a network by sending a high volume of traffic to it. The attacks slowed down CAV workload computations by keeping SoC components busy, negating performance gains achieved by the heterogeneous design. The attacks are:

(1) **FFT:** FFT tiles are targeted by inserting 1000 additional FFT tasks, causing traffic on all FFT accelerators and slowing down the CAV's distance calculations. Figure 4 shows an example of traffic caused by the FFT attack.
(2) **Viterbi:** Viterbi tiles are targeted similarly to the FFT attack with 1000 additional randomly sized Viterbi message decoding tasks sent to all available Viterbi accelerators, slowing down decoding for incoming messages.
(3) **FFT + Viterbi:** Both FFT and Viterbi tiles are targeted by sending out 1000 tasks of each kind to all available accelerators, slowing down both CAV workload tasks.
(4) **Memory:** The memory tile is targeted with 500,000 requests to random addresses in a space 2X the size of the LLC. Frequent cache misses occur, slowing down actual memory requests.

4.3 Evaluation Methods

We conducted both offline and online anomaly detection experiments with SoC A for our case study. In offline experiments, we compared the selected semi-supervised models to each other and to related supervised models. We also experimented with increased message load and traffic variability to test model robustness when detecting Viterbi-based attacks in highly variable settings. Based on offline studies, we chose one of the best performing models to deploy on SoC A for online attack detection experiments on the FPGA. NoC-based counters were sampled every 100ms and the data was input to the deployed model for prediction. NoC counter data from initiating and ending the CAV workload was retained for SoC A datasets so that anomaly detection could be performed during these parts of the software during online testing.

As SoC B offered multiple CPU cores, it was used for localization tests to determine how well SoCurity's NoC counters could aid in pinpointing detected anomalous activity in the SoC. The initial and end data from the CAV workload software was not included in the offline tests for SoC B since this data is not indicative of the hardware activity that would otherwise be occurring on a CAV SoC. Due to this difference in data inclusion, there were fewer samples in the training and testing datasets for SoC B than those used for SoC A.

4.3.1 Training Data. Training data was collected by running the workload (Section 4.1.1) three times on each SoC implementation for 300 time steps each, i.e., 900 total time steps. Each time step took about 150ms on SoC A while all NoC counter data was collected every 100ms.[2] The training dataset for SoC A had 1,094 samples, each with 355 features from counters (Section 4.1.2). Mean and standard deviation for selected features are provided in Table 1, highlighting the features experiencing the most variability for this workload. The training dataset for SoC B had 725 samples, each with 395 features from counters.

4.3.2 Testing Data. For offline tests, test data sets were collected for each SoC with five runs of the CAV workload and five runs of each DoS attack executed during the CAV workload. The workload ran for 300 time steps (i.e., 1,500 total time steps for each test dataset) with counter data

[2]This was the most frequent data collection possible while the CAV workload ran on the single-core setup in SoC A. To keep experiments consistent, we retained this frequency for experiments run on SoC B.

Table 1. Six Features for Anomaly Detection in **SoC A**

Counter Type	Feature Descriptions	Mean/SD of Training Data
General	• CPU NoC packets for memory-mapped register accesses, I/O reads/writes, interrupt requests	0.03/0.03
Memory	• Off-chip memory accesses • DMA requests • Coherence requests	0.27/0.07 0.38/0.34 0.32/0.07
Accelerator	• FFT total cycles • Viterbi total cycles	0.27/0.27 0.19/0.26

Mean and standard deviation values for each normalized feature in the training data are also provided.

Table 2. 13 Features for Detection and Localization in **SoC B**

Counter Type	Feature Descriptions
General	• CPU_1 NoC packets for memory-mapped register accesses, I/O reads/writes, interrupt requests • CPU_2 NoC packets for memory-mapped register accesses, I/O reads/writes, interrupt requests • I/O tile NoC packets for memory-mapped register accesses, I/O reads/writes, interrupt requests • Memory tile NoC packets for memory-mapped register accesses, I/O reads/writes, interrupt requests
Memory	• Coherence requests • DMA requests • L2 hits for CPU_1 • L2 hits for CPU_2
Accelerator	• FFT_1 invocations • FFT_2 invocations • $Viterbi_1$ invocations • $Viterbi_2$ invocations • $Viterbi_3$ invocations

collected every 100ms. Test sets had 3,684 to 4,932 samples for SoC A, and 2,453 to 3,023 samples for SoC B.

4.3.3 Preprocessing Datasets. SoC A was equipped with 355 total NoC counters while SoC B had 395 (Section 4.1.2). However, using all these counters as features for our selected models would cause an unnecessary performance hit to prediction with little added benefit to detection. Many counters correlated heavily and provided data that can be inferred from related counters. We thus reduced our feature set by: (1) selecting the k best features based on variance to find the subset of features that were most impacted by the DoS attacks, (2) using correlation matrices to select counters with unique data that is not derivative of other selected counters, and (3) combining sets of counters to create new informative features, e.g., summing usage cycles for all accelerators of a type on SoC A. While we performed the last reduction step for SoC A, we left the individual accelerator counters for SoC B as is. Since these counters would still be collected individually before they are combined, the individual counters are more informative for localizing anomalous activity to specific accelerators in the localization experiments done on SoC B. Our final datasets consisted of six features for SoC A (Table 1) and 13 features for SoC B (Table 2), making prediction quite fast.

4.3.4 Ground Truth Labels for Test Sets. Model evaluations in our offline experiments hinged on ground truth labels for our test data. Although manual labeling is the most accurate approach, it can be time-consuming for large datasets. Instead, we automated the labeling process by using the unsupervised k-means clustering algorithm to approximately separate data during DoS attacks from benign regular data and label them accordingly. This labeling approach improved analysis efficiency and generally upheld model evaluation accuracy.

Table 3. Average DoS Attack Durations, Average Total Durations of CAV Workload During Attacks, and Impact of Each Attack (Higher Values Indicate Higher Impact)

Attack	Attack Time (s)	Total Time (s)	Impact
No attack	–	46.56	–
FFT	11.55	55.59	5.75
Viterbi	94.13	146.46	4.89
FFT + Viterbi	104.13	155.74	5.00
Memory	40.89	61.71	2.00

Absolute times reflect 300 time steps of the workload running on a 78MHz FPGA for SoC A. Runtimes for SoC B follow similarly.

5 Results

Each DoS attack impacted the CAV workload running on the SoCs. For SoC A, Table 3 gives each attack duration, the time to run the CAV workload for 300 time steps during each attack, and an impact metric for each attack's overall effect on the workload. SoC B runtimes follow similarly. As each attack varies in length, overall workload slowdown does not provide a good comparison for the impact each attack had on the CAV workload. Instead, we include an impact metric for fair comparison. Impact is quantified as the ratio of the workload's slowdown ($\frac{t_{total}}{t_{no_attack}}$) to the proportion of the run during which the attack is executed ($\frac{t_{attack}}{t_{total}}$). In this section, we present results from our anomaly detection and localization experiments using these attacks on SoC A and B.

5.1 Comparing Models for DoS Attack Detection

To evaluate anomaly detection performance of each semi-supervised model on SoC A, we counted **true positive** (**TP**), **false positive** (**FP**), **true negative** (**TN**), and **false negative** (**FN**) predictions for the test datasets, where the positive class is anomalies and predictions were compared against automated ground truth labels (Section 4.3.4). When tuning model parameters, we prioritized low FNs (<10% on SoC A) to minimize missed anomalies and low FPs to avoid system resource costs.

Figure 5 presents FNs, TPs, FPs, and TNs for each model and DoS attack on SoC A. We found attacks with higher *impact* on the CAV workload (Table 3) were more easily detected. All models detected FFT attacks best; there were no FNs. OCSVM had the lowest FNs and FPs for Viterbi and FFT + Viterbi attacks. All models had high FPs for memory attacks which had the least impact and thus less accurate clustered ground truth labels than other attacks. In online tests (Section 5.3), far fewer FPs occurred for memory attacks. Overall, all models were highly effective, but OCSVM best distinguished variable message traffic decoding from Viterbi-based attacks in SoC A.

Semi-Supervised vs. Supervised Models: To underscore our detection models' enhanced ability to detect unknown attacks compared to supervised models, we compared the semi-supervised models with their supervised variants, using similar parameters for training. We examined KNN, SVM, and random forests models, training them with anomalous data from FFT attacks, and compared their abilities to detect Viterbi attacks against the semi-supervised models.

Figure 6 presents **receiver operating characteristic** (**ROC**) curves for SoC A comparing Viterbi attack detection with the supervised models against our detection system's semi-supervised models that were only trained with CAV workload data. False positive rate (FPR: $\frac{FP}{FP+TN}$, lower is better) is on the x-axis and recall ($\frac{TP}{TP+FN}$, higher is better) is on the y-axis. **Areas under each model's curve** (**AUCs**) are given in the legend. AUCs closest to 1 have the best model performances. We found KNN to be the least effective model with a 0.500 AUC (equivalent to random guessing), while its semi-supervised counterparts OCNN and LOF showed robust performance with AUCs

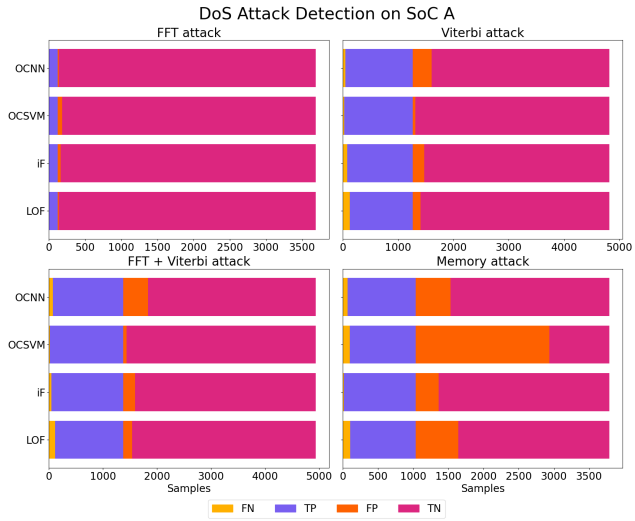


Fig. 5. FN, TP, FP, TN for each model and attack. Low FN and FP are better. All models performed well on FFT attacks. OCSVM performed best for Viterbi and FFT + Viterbi attacks. All had high FPs for memory attacks.

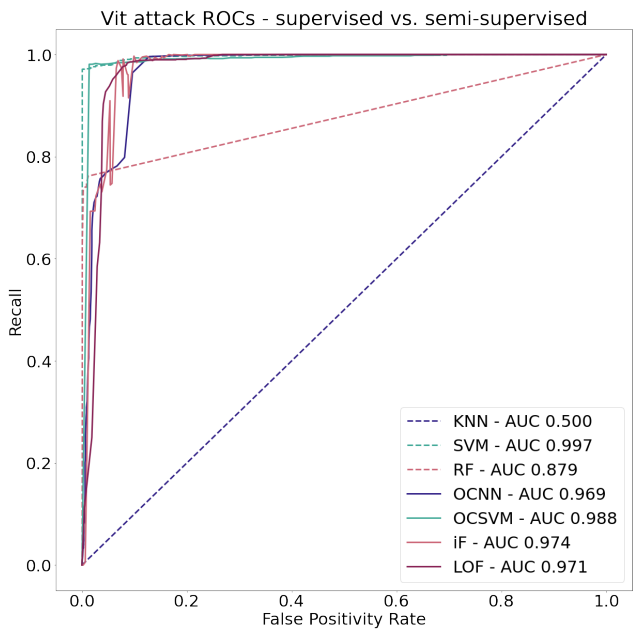
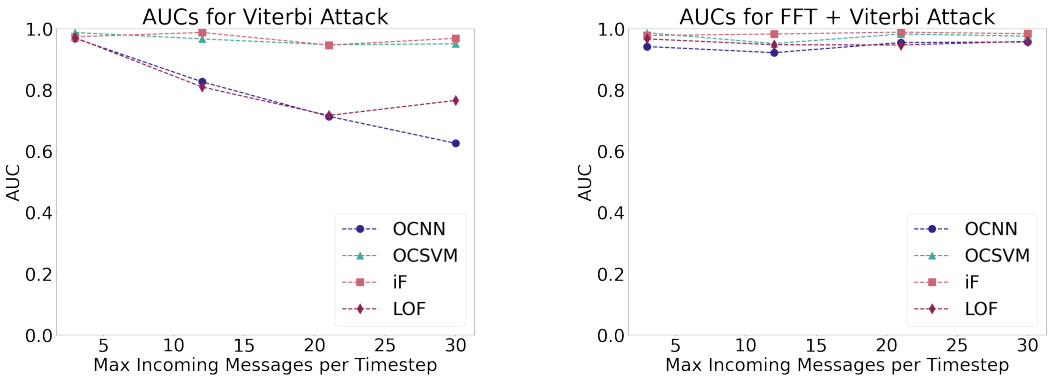


Fig. 6. ROC curves for viterbi attack detection with SoC A using supervised (dashed) and semi-supervised (solid) models, with AUCs (areas under curves, higher is better) in the legend. Most semi-supervised detection models outperformed their supervised counterparts that were trained on FFT attack data.

of 0.969 and 0.971, respectively. Random forests had a 0.879 AUC, but it was surpassed by its semi-supervised counterpart iForest which had a 0.974 AUC. SVM slightly outperformed its semi-supervised counterpart OCSVM, but both had the two largest AUCs at 0.997 and 0.988, respectively. These results show that semi-supervised ML models used in our anomaly detection system are



(a) Viterbi Attack (b) FFT + Viterbi Attack

Fig. 7. AUC impact for each model when incoming message traffic variability of CAV workload increases (higher is better). OCSVM and iForest maintain high AUCs for both attacks.

superior at detecting "unknown" attacks compared to supervised models that are trained with data from similar attacks.

Storage Requirements: While iForest and LOF models had comparable detection accuracy to OCSVM, iForest required significantly more storage at about 807.3KB and LOF required 316.0KB. The OCNN model required 187.0KB. We found the OCSVM model required the smallest amount of storage at just 2.9KB, making it lightweight and ideal for fast detection.

5.2 SoCurity Robustness

We stress tested the SoCurity detection system to see how Viterbi-based attack detection is impacted under conditions with much higher incoming message traffic for the CAV workload, simulating extreme urban settings with large CAV presence. We randomized message traffic with up to 3, 12, 21, and 30 incoming messages of random size per time step on SoC A. Beyond three messages, Viterbi accelerators faced increased task loads. Figure 7 shows AUCs for detecting Viterbi and FFT + Viterbi attacks under these conditions for each semi-supervised model. For Viterbi attacks (Figure 7(a)), OCSVM and iForest maintained high AUCs with increasing variability, while OCNN and LOF had decreased AUC with higher variability. As distance-based models, OCNN and LOF struggled to distinguish between high Viterbi activity due to incoming message traffic and the Viterbi attack. In contrast, OCSVM and iForest incorporate feature relations for a stronger model. For FFT + Viterbi attacks (Figure 7(b)), all models maintained high accuracy. OCNN and LOF distinguished anomalous activity better with the presence of increased FFT activity from the attack, tracking with their high detection performance for the FFT-only attack (Figure 5). Overall, using OCSVM or iForest for detection, SoCurity is robust to highly variable swarm communication conditions processed by the CAV workload.

5.3 Fast and Lightweight SoCurity in Action on the CAV SoC

We selected OCSVM for online experiments as it was the most lightweight model, with two orders of magnitude less storage required than others. It also performed well across all offline tests on SoC A and was robust against higher variability introduced to the base workload. We statically trained the model and deployed it on the FPGA with m2cgen [71]. When run on the 78MHz FPGA, the OCSVM model had a 320µs prediction time (32µs on an ASIC [31]). This *prediction* time better represents *detection* time for an SoC with a dedicated SoCurity analyzer unit. In this study, the

Table 4. Average Detection Cycles to Flag the Attack (Lower is Better), Accuracy (Higher is Better), and FPR ($\frac{FP}{TN+FP}$, Lower is Better) for Each Attack on SoC A on the FPGA

Attack	Detection Cycles	Accuracy	FPR
FFT	0.96	0.96	0.04
Viterbi	2.82	0.87	0.05
FFT + Viterbi	1.06	0.87	0.06
Memory	2.22	0.60	0.05

Accuracy correlates with impact (Table 3).

SoCurity analyzer shared the single core in SoC A with the CAV workload and the attacks, limiting detection cycles to every 100ms.

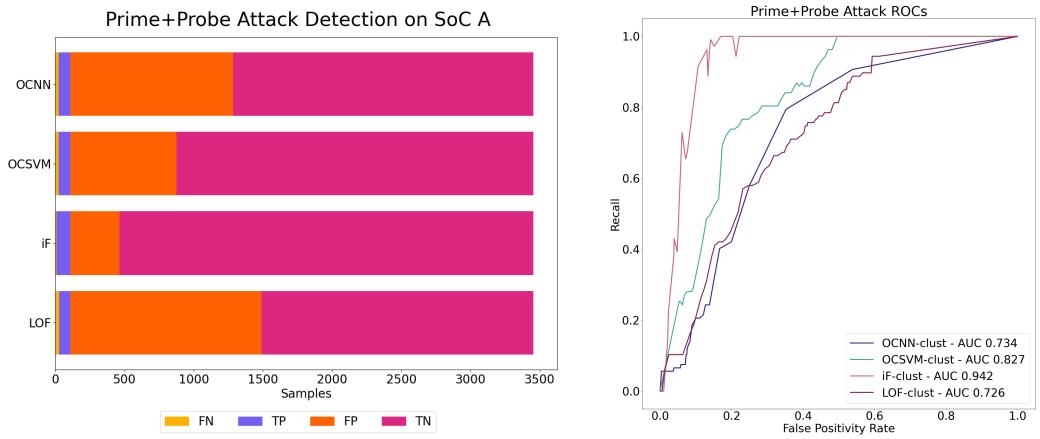
Table 4 presents average detection cycles required to flag attacks after they are launched (where cycle 0 is the window in which the attack is launched), accuracy ($\frac{TP+TN}{TP+FP+TN+FN}$), and FPRs ($\frac{FP}{TN+FP}$) for each DoS attack on the FPGA. TP and FN were counted as prediction cycles during each attack where counters were predicted anomalous and benign, respectively. TN and FP were counted as the cycles before and after each attack where counters were predicted benign and anomalous, respectively. The OCSVM model had low FPRs for each attack and high accuracy for accelerator-based DoS attacks. On average, the FFT attack was detected in the same cycle window in which the attack was launched (average of <1 detection cycles). Due to the memory attack's lower impact (Table 3), some counter cycles were not as recognizably anomalous so the model had lower detection accuracy. Nonetheless, the attack was detected within a few detection cycles, so this anomalous behavior would be quickly investigated by the localization engine. These results demonstrate SoCurity's strong abilities for effective anomaly detection in heterogeneous SoCs.

5.4 Detecting the *Prime+Probe* Cache Side-Channel Attack

In addition to availability attacks like DoS attacks, SoCurity can detect other attack vectors that impact the resource usage on an SoC. To demonstrate this, we deployed a *Prime+Probe* cache side-channel attack [50] on the CAV workload running on SoC A and tested whether the NoC counter data was impacted enough to detect the attack with SoCurity. The Prime+Probe attack *primes* the L1 cache by filling each cacheline with some data. It then waits for several cycles, allowing the victim CAV workload to run for some time. The attack then *probes* the cache by accessing some data on each cacheline and timing each access. Since the attacker prefilled each cacheline during the priming phase, it can determine which cachelines were touched by the victim workload based on its own cache misses. The attack repeats this process periodically to uncover confidential information such as secret keys.

When run alongside the CAV workload, the Prime+Probe workload has an average 2.28 impact on the workload, running for an average of 51.82s and extending the total workload runtime to 75.80s. The impact of the Prime+Probe attack is similar to the Memory DoS attack, yet results in less inter-component communications through the NoC as its memory footprint largely stays within the L1 cache on the Ariane core, unlike the Memory DoS attack which extends past the LLC (Section 4.2). Nonetheless, SoCurity is able to detect the Prime+Probe attack within two detection cycles.

Figure 8 summarizes the detection results for the Prime+Probe attack tests. The models were tuned for effective attack detection (<25% FN), but faced higher FPs than other tested attacks. The iForest model was still able to effectively detect the Prime+Probe attack with few FNs and FPs. These results show that SoCurity can capture a variety of attacks. For best performance, SoCurity should



(a) FN, TP, FP, and TN for each model detecting the Prime+Probe attack. Low FN and FP are better. iForest performed the best with the lowest FNs and FPs.

(b) AUC for each model detecting the Prime+Probe attack. iForest had the highest AUC at 0.942.

Fig. 8. SoCurity detection results for the Prime+Probe attack.

be deployed as an ensemble of detection models that can detect a variety of attacks, additionally improving the likelihood of novel attack detection. Furthermore, this implementation method provides protection against evasive attacks [30, 33], as discussed in Section 6.

5.5 DoS Attack Localization on the Multicore SoC B

5.5.1 Detection. The multicore design of SoC B was used to test localization capability with SoCurity's NoC counters as the multiple cores on the SoC provide greater variability in the location of an attack. NoC counter data that is flagged by the anomaly detection engine is sent to the localization engine. First, we needed to evaluate anomaly detection performance of the semi-supervised models on DoS attacks running on SoC B. Figure 9 shows the FNs, TPs, FPs, and TNs resulting from running detection with each model on each attack on counter data collected from SoC B. These tests revealed LOF to perform the best overall for detecting each attack with low FNs and FPs. We thus use the LOF model in localization tests for SoC B in this section.

5.5.2 Rule Extraction. To test localization ability on the NoC counter data collected from SoC B, we used a rule extraction algorithm inspired by [7], with benign and anomalous data labels outputted by the LOF model to recursively cluster the data with hypercubes until they contain only benign data. The hypercube bounds can provide IF-THEN rules that check whether a sample falls within the feature bounds of benign training data. We found that in the SoC B datasets, the minimum and maximum values of each feature in the training data could be used as the bounded rules. To *interpret* the source and victim from the anomalous NoC counter data, the impact of more than one counter must be considered. For example, the FFT + Viterbi DoS attack impacts all FFT and Viterbi accelerators and may also impact the memory tile. Based on the components in SoC B, we derived one interpretation for anomalous activity that can result in anomalous NoC counter data, and found that it aligns well with the tested DoS attacks.

Figure 10 shows a high level diagram of our localization algorithm based on our derived interpretations. The NoC counters collected for a sample that is flagged anomalous by the detection system is input to this algorithm and steps through the checks in the blue boxes. The green boxes indicate when anomalous activity is localized. For example, if the **DMA requests** counters are anomalous,

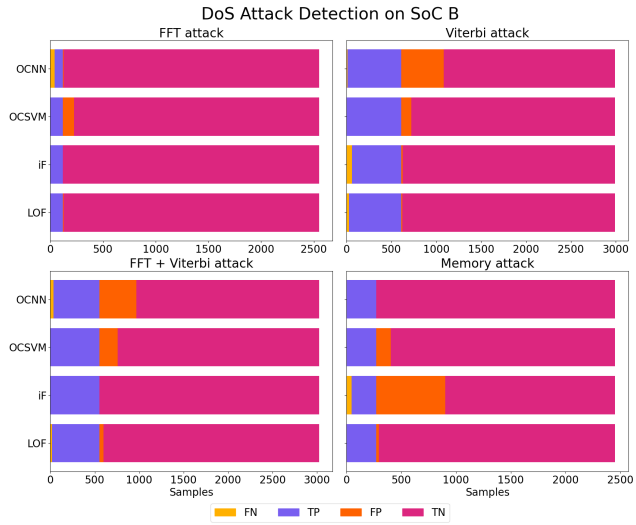


Fig. 9. FN, TP, FP, TN for each model and attack. Low FN and FP are better. All models performed well on FFT attacks. LOF performed well for all attacks.

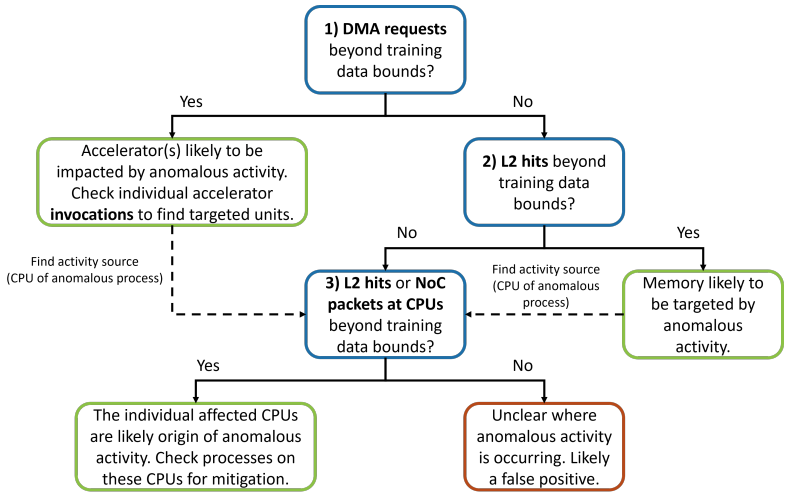


Fig. 10. Localization algorithm for NoC counter values residing outside of training data bounds.

there may be accelerators impacted, as DMA requests are primarily made by accelerators. The impacted accelerators can then be determined by checking each accelerator's **invocation** counter. This step makes it possible to uncover whether more than one accelerator is being impacted. To determine which core(s) an attack may be launching from, the **L2 hits** at each CPU and the **NoC packets injected at each CPU** are checked. This may lead to the discovery of one or more CPUs that are involved in running the anomalous activity. If none of these anomalous activity interpretations are determined, it is unclear where the anomalous activity is occurring and it may be an FP (red box). Though we did not observe any consecutive NoC counter samples that were labeled FPs in our study, in practice, such consecutive FPs should be further investigated to determine if they

Table 5. Localization Results

Attack Dataset	Correct Attack Localization	Other Anomaly Localized	Undetermined
FFT	0.94	0.03	0.03
Viterbi	0.88	0.11	0.01
FFT + Viterbi	0.95	0.03	0.02
Memory	0.99	0.00	0.01

The localization engine pinpointed the attack targets with high accuracy. Other anomalous activity resulting from attack impacts were also localized. In a few remaining cycles, the localization engine did not find any particular components where anomalous activity was prevalent.

are truly FPs. This follow-up investigation can be executed by a mitigation system implemented within the SoCurity analyzer, or by a trusted hypervisor or OS that can verify whether all running processes are known and benign.

5.5.3 Localization Results. Table 5 summarizes results from running the localization algorithm (Figure 10) on each sample labeled anomalous by the LOF model during each DoS attack on the multicore SoC B. The **Correct Attack Localization** column gives the portion of samples that were interpreted to be causing anomalous activity at the correct target locations of each attack (e.g., the FFT accelerator(s) for the FFT attack). Most samples were accurately localized to the correct victim units for all attacks. The algorithm also determined the CPU source of the anomalous activity for each sample. The **Other Anomaly Localized** column gives the proportion of samples where the algorithm determined other non-victim SoC components were affected (e.g., increased memory tile traffic during an accelerator attack). The **Undetermined** column indicates the portion of samples where the localization algorithm could not establish a clear anomalous location. Few samples fell in this category and further analysis revealed that they aligned with the FPs detected during the LOF model evaluation (Figure 9). Overall, these results show that the SoCurity counters enable anomalous activity to be accurately localized in the SoC with our devised localization algorithm, such that precise mitigation measures can be taken to preserve system performance where possible.

5.6 Putting It All Together

The presented results show that SoCurity's NoC monitoring system can be leveraged to effectively detect anomalous activity for hardware-level security systems. SoCurity's counters can then be used by a localization engine to accurately locate anomalous activity in parts of an SoC (e.g., specific accelerator, source CPU), and aid threat diagnosis and mitigation while preserving performance.

6 Discussion

Mitigation: Once anomalous activity is detected and localized, any threats in the SoC must be mitigated. Although outside the scope of this article, data provided by SoCurity's counters, detection, and localization can aid a trusted supervisor or OS to recover impacted SoC components without requiring full-system recovery measures. Mitigation techniques may include (1) searching for unknown processes running on impacted cores, (2) comparing process resource usage to anomalous SoC component usage activity detected and localized with SoCurity counters, (3) ending or quarantining any such anomalous processes found running in the SoC, and (4) rebooting any impacted SoC components to end any ongoing anomalous activity occurring in them. This can greatly improve safety-critical system performance and availability when under attack. The

techniques discussed in this article provide examples of SoCurity defense systems that we found to be successful, but more sophisticated SoC security solutions may also be developed with SoCurity counters using more powerful ML models or time-series methods.

Scalability: Designing larger SoCs with SoCurity requires more NoC counters throughout. If counter monitoring is inefficient for a single detection unit, detection can be parallelized across multiple units each monitoring a subset of tiles. This approach maintains regional NoC traffic knowledge throughout the SoC without requiring a single unit to process all data.

Evasive Attacks: Since SoCurity performs continuous hardware-level monitoring, we have found it to be quite sensitive and able to detect even small attacks that cause negligible runtime impact on the CAV workload. As described in the threat model (Section 3.1), SoCurity is primarily designed to protect heterogeneous SoCs against threats that can compromise the performance gains achieved from their designs. In order for an evasive attack to impact a workload's performance while mimicking benign behavior, it must also be able to suppress the real workload from executing. To withstand such attacks, SoCurity can be built by training several detection models and randomly selecting from them for prediction [30, 33].

SoCurity for Other Detection: The SoCurity approach is simple and generalizable, so it can be used to detect SoC threats beyond DoS attacks. For example, SoCurity counters can detect *black hole router* attacks launched by hardware trojans, where one node in the NoC drops all received packets, creating a "black hole" in the network [17]. Side-channel attacks like prime-and-probe [50] that repeatedly access shared resources to learn secrets can be detected, as demonstrated in Section 5.4. Prior work also showed that these attacks impact hardware counters [18]. Likewise, other attacks that may impact resource usage on the system such as code injection attacks, fault injection attacks, and buffer overflow attacks may be detected by SoCurity when they result in altered NoC interactions.

SoCurity counters can also be expanded to support power token monitoring [60] and detect if malicious actors interject and manipulate token flow, which could cause imbalanced power and overheating in the SoC. Finally, SoCurity can be used for hardware reliability. Resources may fail to operate as expected, causing slowdowns that are reflected in SoCurity counters. One example is **silent data corruption** (**SDC**), in which a CPU produces incorrect computational results [22]. Though SDC errors are not identifiable by existing error handling systems, it may be possible to detect NoC communication pattern changes caused by SDC in an SoC with SoCurity counters. We leave these SoCurity-based attack and reliability detection systems for future work.

7 Related Work

ML for Anomaly and Malware Detection: Over the past decade, interest has grown in malware detection using ML models [5, 26, 27, 38, 55]. However, many of these works are not sufficient for modern heterogeneous SoCs as they are either (1) focused on CPU-based system activity (e.g., CPU hardware performance counters), or (2) not designed for future novel attack detection, an important consideration due to rapid changes in modern system designs that more easily introduce new vulnerabilities. Most of these works use supervised ML models that are trained on both benign and malicious data to detect known attacks, so they are not well-designed for detecting novel unknown threats [2, 18, 34, 37, 43, 52]. Bourdon et al. used unsupervised models for anomaly detection in massively deployed homogeneous smart industrial devices [10]. Some works looked at semi-supervised anomaly detection models that do not rely on known attack data [2, 35, 65], but these are based on CPU-specific performance counter data. Other works looked at attack detection based on CPU activity that is not collected with hardware performance counters, such as Intel's Threat Detection Technology [29] and MADAM for malware detection in Android systems [21].

Attack Detection in NoC-based SoCs: As discussed in [15] and [64], several works have explored security measures for NoC-based SoCs in which **CPU** performance counters are not sufficient. [52] demonstrated this weakness in CPU performance counters by evaluating ML-based malware detection with them on SoC cores and found high FPRs averaging 11.3%. Furthermore, [20] showed that CPU counter-based detectors can be reverse-engineered with software tools like `perf`. SoCurity's NoC counters are resilient against such adversarial attacks as they are inaccessible to software-level tools. Many prior works similarly monitored data based on NoC activity for attack detection, but none sufficiently accounted for NoC activity in heterogeneous SoCs with non-CPU elements. [19] devised a randomized routing system to thwart hardware trojans embedded within a third-party NoC. While effective, this type of hardware trojan is not part of our threat model as SoCurity is built into NoC routers and thus should not be a third-party component of the target SoC. [36, 58], and [52] looked at anomaly and malware detection with basic NoC packet transmission data like source core, destination core, and distance between them. [58] proposed a hierarchical security monitoring approach for multicore SoCs, where NoC packet information is stored in a security table and checked for authentication and access control. [52] studied cache coherence NoC packets, similar to [63], which looked at DoS attack detection in a multicore SoC. However, non-CPU components in heterogeneous SoCs may rely on DMAs with the memory controller to retrieve data, which these works do not cover. SoCurity thus provides more coverage for SoCs than prior works, while also retaining simplicity as information extraction from packets like in [58] and [63] is not needed. While some works explored attack detection on SoCs with non-CPU components, they lacked broad applicability. Ref. [23] proposed security policies on permitted NoC packet transmissions that are non-trivial to define. Ref. [14] adopted an impractical DoS attack detection and localization approach for SoCs, relying on NoC traffic latency predictability which is susceptible to high FPRs. Similarly, [56] used NoC traffic latency to detect bandwidth denial attacks by a malicious NoC.

8 Conclusion

In this article, we presented SoCurity, a design approach for enhancing security of SoCs by building hardware activity monitoring at the NoC-level with NoC-based hardware counters that track interactions between SoC components. Using SoCurity, we developed a fast and lightweight anomaly detection and localization system for detecting and locating activity impacting availability of SoC components. Our proposed system does not rely on attack data for detection so it is able to detect and locate both existing availability attacks and future novel attacks. In particular, the anomaly detection system is developed using an adaptive approach based on semi-supervised ML models. We demonstrated SoCurity and the proposed detection and localization system with a case study on real SoC implementations for a CAV system. Our techniques resulted in highly accurate (up to 96%) and fast (~30µs) detection and highly reliable (up to 99%) localization of DoS attacks on the CAV SoC.

Acknowledgments

This work was supported in part by National Science Foundation (NSF) award No. 1763838, and based on research sponsored by the Air Force Research Laboratory (AFRL) and Defense Advanced Research Projects Agency (DARPA) under agreement FA8650-18-2-7862. In addition, Martonosi received separate NSF support while serving at NSF as an IPA rotator.[3]

[3]The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of NSF, AFRL and DARPA or the U.S. Government. Distribution Statement "A": Approved for Public Release, Distribution Unlimited.

References

[1] Saeed Ahmed, Youngdoo Lee, Seung Ho Hyun, and Insoo Koo. 2019. Unsupervised machine learning-based detection of covert data integrity assault in smart grid networks utilizing isolation forest. *IEEE Trans. Inf. Forensics Secur.* 14, 10 (2019), 2765–2777.

[2] Manaar Alam, Sarani Bhattacharya, Debdeep Mukhopadhyay, and Sourangshu Bhattacharya. 2017. Performance counters to rescue: A machine learning based safeguard against micro-architectural side-channel-attacks. *International Association for Cryptologic Research (IACR) Cryptol. ePrint Arch.* (2017).

[3] Manos Antonakakis, Tim April, Michael D. Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, et al. 2017. Understanding the Mirai Botnet. In *26th USENIX Security Symposium*, Engin Kirda and Thomas Ristenpart (Eds.). USENIX Association.

[4] Alejandro Barredo Arrieta, Natalia Díaz Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, et al. 2020. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion* 58 (2020), 82–115.

[5] Michael Bailey, Jon Oberheide, Jon Andersen, Zhuoqing Morley Mao, Farnam Jahanian, and Jose Nazario. 2007. Automated classification and analysis of internet malware. In *Recent Advances in Intrusion Detection, 10th International Symposium, RAID (Lecture Notes in Computer Science)*, Christopher Krügel, Richard Lippmann, and Andrew J. Clark (Eds.). Springer.

[6] Pete Bannon, Ganesh Venkataramanan, Debjit Das Sarma, and Emil Talpes. 2019. Computer and redundancy solution for the full self-driving computer. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE.

[7] Alberto Barbado, Óscar Corcho, and Richard Benjamins. 2022. Rule extraction in unsupervised anomaly detection for model explainability: Application to OneClass SVM. *Expert Syst. Appl.* 189 (2022), 116100.

[8] Swarup Bhunia, Michael S. Hsiao, Mainak Banga, and Seetharam Narasimhan. 2014. Hardware trojan attacks: Threat analysis and countermeasures. *Proc. IEEE* 102, 8 (2014), 1229–1247.

[9] Shekhar Borkar and Andrew A. Chien. 2011. The future of microprocessors. *Commun. ACM* 54, 5 (2011), 67–77.

[10] Malcolm Bourdon, Pierre-François Gimenez, Eric Alata, Mohamed Kaâniche, Vincent Migliore, Vincent Nicomette, and Youssef Laarouchi. 2020. Hardware-performance-counters-based anomaly detection in massively deployed smart industrial devices. In *19th IEEE International Symposium on Network Computing and Applications, NCA*. IEEE.

[11] Leo Breiman. 2001. Random forests. *Mach. Learn.* 45, 1 (2001), 5–32.

[12] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. 2000. LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, Weidong Chen, Jeffrey F. Naughton, and Philip A. Bernstein (Eds.). ACM.

[13] Varun Chandola, Arindam Banerjee, and Vipin Kumar. 2009. Anomaly detection: A survey. *ACM Comput. Surv.* 39, 12 (2009), 4510–4523.

[14] Subodha Charles, Yangdi Lyu, and Prabhat Mishra. 2020. Real-time detection and localization of distributed DoS attacks in NoC-based SoCs. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 39, 12 (2020), 4510–4523.

[15] Subodha Charles and Prabhat Mishra. 2022. A survey of network-on-chip security attacks and countermeasures. *ACM Comput. Surv.* 54, 5 (2022), 1–36.

[16] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Mach. Learn.* 20, 3 (1995), 273–297.

[17] Luka Daoud and Nader Rafla. 2019. Analysis of black hole router attack in network-on-chip. In *62nd IEEE International Midwest Symposium on Circuits and Systems, MWSCAS*, Hoi Lee and Randall L. Geiger (Eds.). IEEE.

[18] John Demme, Matthew Maycock, Jared Schmitz, Adrian Tang, Adam Waksman, Simha Sethumadhavan, and Salvatore J. Stolfo. 2013. On the feasibility of online malware detection with performance counters. In *The 40th Annual International Symposium on Computer Architecture, ISCA'13*, Avi Mendelson (Ed.). ACM.

[19] Abhijitt Dhavlle, M. Meraj Ahmed, Naseef Mansoor, Kanad Basu, Amlan Ganguly, and Sai Manoj Pudukotai Dinakarrao. 2023. Defense against on-chip trojans enabling traffic analysis attacks based on machine learning and data augmentation. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 42, 12 (2023), 4681–4694.

[20] Sai Manoj Pudukotai Dinakarrao, Sairaj Amberkar, Sahil Bhat, Abhijitt Dhavlle, Hossein Sayadi, Avesta Sasan, Houman Homayoun, and Setareh Rafatirad. 2019. Adversarial attack on microarchitectural events based malware detectors. In *Proceedings of the 56th Annual Design Automation Conference 2019, DAC*. ACM.

[21] Gianluca Dini, Fabio Martinelli, Andrea Saracino, and Daniele Sgandurra. 2012. MADAM: A multi-level anomaly detector for android malware. In *Computer Network Security—6th International Conference on Mathematical Methods, Models and Architectures for Computer Network Security, MMM-ACNS (Lecture Notes in Computer Science)*, Igor V. Kotenko and Victor A. Skormin (Eds.). Springer.

[22] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent data corruptions at scale. *arXiv preprint arXiv:2102.11245* (2021).

[23] Leandro Fiorin, Gianluca Palermo, and Cristina Silvano. 2008. A security monitoring service for NoCs. In *Proceedings of the 6th International Conference on Hardware/Software Codesign and System Synthesis, CODES+ISSS*, Catherine H. Gebotys and Grant Martin (Eds.). ACM.

[24] Evelyn Fix and Joseph Lawson Hodges. 1989. Discriminatory analysis. Nonparametric discrimination: Consistency properties. *International Statistical Review/Revue Internationale de Statistique* 57, 3 (1989), 238–247.

[25] Fei Gao, Ting-Jung Chang, Ang Li, Marcelo Orenes-Vera, Davide Giri, Paul J. Jackson, August Ning, Georgios Tziantzioulis, Joseph Zuckerman, Jinzheng Tu, et al. 2023. DECADES: A 67mm^2, 1.46TOPS, 55 Giga cache-coherent 64-bit RISC-V instructions per second, heterogeneous manycore SoC with 109 tiles including accelerators, intelligent storage, and eFPGA in 12nm FinFET. In *IEEE Custom Integrated Circuits Conference, CICC*. IEEE.

[26] Dragos Gavrilut, Mihai Cimpoesu, Dan Anton, and Liviu Ciortuz. 2009. Malware detection using machine learning. In *Proceedings of the International Multiconference on Computer Science and Information Technology, IMCSIT*. IEEE.

[27] Daniel Gibert, Carles Mateu, and Jordi Planes. 2020. The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *Journal of Network and Computer Applications* 153 (2020), 102526.

[28] Qi Guoqing. 2004. High accuracy range estimation of FMCW level radar based on the phase of the zero-padded FFT. In *Proceedings 7th International Conference on Signal Processing, ICSP'04*. IEEE.

[29] Intel. 2023. Detect Ransomware and Other Advanced Threats with Intel® Threat Detection Technology. Retrieved from https://www.intel.com/content/www/us/en/architecture-and-technology/threat-detection-technology-brief.html

[30] Md. Shohidul Islam, Khaled N. Khasawneh, Nael B. Abu-Ghazaleh, Dmitry Ponomarev, and Lei Yu. 2022. Efficient hardware malware detectors that are resilient to adversarial evasion. *IEEE Trans. Computers* 71, 11 (2022), 2872–2887.

[31] Tianyu Jia, Paolo Mantovani, Maico Cassel dos Santos, Davide Giri, Joseph Zuckerman, Erik Jens Loscalzo, Martin Cochet, Karthik Swaminathan, Gabriele Tombesi, Jeff Jun Zhang, et al. 2022. A 12nm agile-designed SoC for swarm-based perception with heterogeneous IP blocks, a reconfigurable memory hierarchy, and an 800MHz multi-plane NoC. In *48th IEEE European Solid State Circuits Conference, ESSCIRC*. IEEE.

[32] Shehroz S. Khan and Amir Ahmad. 2018. Relationship between variants of one-class nearest neighbors and creating their accurate ensembles. *IEEE Trans. Knowl. Data Eng.* 30, 9 (2018), 1796–1809.

[33] Khaled N. Khasawneh, Nael B. Abu-Ghazaleh, Dmitry Ponomarev, and Lei Yu. 2017. RHMD: Evasion-resilient hardware malware detectors. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO*. Hillery C. Hunter, Jaime Moreno, Joel S. Emer, and Daniel Sánchez (Eds.). ACM.

[34] Jeffrey C. Kimmell, Mahmoud Abdelsalam, and Maanak Gupta. 2021. Analyzing machine learning approaches for online malware detection in cloud. In *IEEE International Conference on Smart Computing, SMARTCOMP*. IEEE.

[35] Prashanth Krishnamurthy, Ramesh Karri, and Farshad Khorrami. 2020. Anomaly detection in real-time multi-threaded processes using hardware performance counters. *IEEE Trans. Inf. Forensics Secur.* 15 (2020), 666–680.

[36] Amey M. Kulkarni, Youngok K. Pino, Matthew French, and Tinoosh Mohsenin. 2016. Real-time anomaly detection framework for many-core router through machine-learning techniques. *ACM J. Emerg. Technol. Comput. Syst.* 13, 1 (2016), 1–22.

[37] Abraham Peedikayil Kuruvila, Sayar Karmakar, and Kanad Basu. 2021. Time series-based malware detection using hardware performance counters. In *IEEE International Symposium on Hardware Oriented Security and Trust, HOST*. IEEE.

[38] Wenke Lee, Salvatore J. Stolfo, and Kui W. Mok. 1999. A data mining framework for building intrusion detection models. In *1999 IEEE Symposium on Security and Privacy*. IEEE Computer Society.

[39] Yuchong Li and Qinghui Liu. 2021. A comprehensive review study of cyber-attacks and cyber security; Emerging trends and recent developments. *Elsevier Energy Reports* 7 (2021), 8176–8186.

[40] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*. IEEE Computer Society.

[41] Luke Lockhart, Paul Harvey, Pierre Imai, Peter Willis, and Blesson Varghese. 2020. Scission: Context-aware and performance-driven edge-based distributed deep neural networks. *arXiv preprint arXiv:2102.11245* (2020), arXiv–2008.

[42] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems*. Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.).

[43] Corey Malone, Mohamed Zahran, and Ramesh Karri. 2011. Are hardware performance counters a cost effective way for integrity checking of programs. In *Proceedings of the Sixth ACM Workshop on Scalable Trusted Computing, STC@CCS*, Yan Chen, Shouhuai Xu, Ahmad-Reza Sadeghi, and Xinwen Zhang (Eds.). ACM.

[44] Paolo Mantovani, Davide Giri, Giuseppe Di Guglielmo, Luca Piccolboni, Joseph Zuckerman, Emilio G. Cota, Michele Petracca, Christian Pilato, and Luca P. Carloni. 2020. Agile SoC development with open ESP : Invited paper. In *IEEE/ACM International Conference On Computer Aided Design, ICCAD*. IEEE.

[45] Raphaël Milan, Lilian Bossuet, Loïc Lagadec, Carlos Andres Lara-Nino, and Brice Colombier. 2023. TrustSoC: Light and efficient heterogeneous SoC architecture, secure-by-design. In *Asian Hardware Oriented Security and Trust Symposium, AsianHOST*. IEEE.

[46] Mohammad Mirza-Aghatabar, Somayyeh Koohi, Shaahin Hessabi, and Massoud Pedram. 2007. An empirical investigation of mesh and torus NoC topologies under different routing algorithms and traffic models. In *Tenth Euromicro Conference on Digital System Design: Architectures, Methods and Tools (DSD)*. IEEE Computer Society.

[47] Prabhat Mishra, Swarup Bhunia, and Mark Tehranipoor. 2017. *Hardware IP Security and Trust*. Springer.

[48] Haydemar Núñez, Cecilio Angulo, and Andreu Català. 2002. Rule extraction from support vector machines. In *10th Eurorean Symposium on Artificial Neural Networks, ESANN*, Michel Verleysen (Ed.).

[49] Marcelo Orenes-Vera, Aninda Manocha, David Wentzlaff, and Margaret Martonosi. 2021. AutoSVA: Democratizing formal verification of RTL module interactions. In *58th ACM/IEEE Design Automation Conference, DAC*. IEEE.

[50] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache attacks and countermeasures: The case of AES. In *Topics in Cryptology - CT-RSA*.

[51] Meltem Ozsoy, Caleb Donovick, Iakov Gorelik, Nael B. Abu-Ghazaleh, and Dmitry V. Ponomarev. 2015. Malware-aware processors: A framework for efficient online malware detection. In *21st IEEE International Symposium on High Performance Computer Architecture, HPCA*. IEEE Computer Society.

[52] Zhixin Pan, Jennifer Sheldon, Chamika Sudusinghe, Subodha Charles, and Prabhat Mishra. 2021. Hardware-assisted malware detection using machine learning. In *Design, Automation & Test in Europe Conference & Exhibition, DATE*. IEEE.

[53] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in python. *J. Mach. Learn. Res.* 12 (2011), 2825–2830.

[54] Marco Túlio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM.

[55] Konrad Rieck, Thorsten Holz, Carsten Willems, Patrick Düssel, and Pavel Laskov. 2008. Learning and classification of malware behavior. In *Detection of Intrusions and Malware, and Vulnerability Assessment, 5th International Conference, DIMVA (Lecture Notes in Computer Science)*, Diego Zamboni (Ed.). Springer.

[56] Rajesh J. S., Dean Michael Ancajas, Koushik Chakraborty, and Sanghamitra Roy. 2015. Runtime detection of a bandwidth denial attack from a rogue network-on-chip. In *Proceedings of the 9th International Symposium on Networks-on-Chip. NOCS*, André Ivanov, Diana Marculescu, Partha Pratim Pande, José Flich, and Karthik Pattabiraman (Eds.). ACM.

[57] Bernhard Schölkopf, Robert C. Williamson, Alexander J. Smola, John Shawe-Taylor, and John C. Platt. 1999. Support vector method for novelty detection. In *Advances in Neural Information Processing Systems 12, [NIPS Conference]*. Sara A. Solla, Todd K. Leen, and Klaus-Robert Müller (Eds.). The MIT Press.

[58] Johanna Sepulveda, Guy Gogniat, Cesar Pedraza, Ricardo Pires, Wang Jiang Chau, and Marius Strum. 2012. Hierarchical NoC-based security for MP-SoC dynamic protection. In *2012 IEEE 3rd Latin American Symposium on Circuits and Systems (LASCAS)*. IEEE.

[59] Biruk B. Seyoum, Davide Giri, Kuan-Lin Chiu, Bryce Natter, and Luca P. Carloni. 2023. PR-ESP: An open-source platform for design and programming of partially reconfigurable SoCs. In *Design, Automation & Test in Europe Conference & Exhibition, DATE*. IEEE.

[60] Parth Shah, Ranjal Gautham Shenoy, Vaidyanathan Srinivasan, Pradip Bose, and Alper Buyuktosunoglu. 2023. TokenSmart: Distributed, scalable power management in the many-core era. *ACM Trans. Archit. Code Optim.* 20, 1 (2023), 1–26.

[61] Stephen Simpson. 2022. Systems on a chip comes to the data center. *McKinsey Digital* (2022). Retrieved from https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/tech-forward/systems-on-a-chip-comes-to-the-data-center

[62] Tyler Sorensen, Aninda Manocha, Esin Tureci, Marcelo Orenes-Vera, Juan L. Aragón, and Margaret Martonosi. 2020. A simulator and compiler framework for agile hardware-software co-design evaluation and exploration. In *IEEE/ACM International Conference On Computer Aided Design, ICCAD*. IEEE.

[63] Chamika Sudusinghe, Subodha Charles, and Prabhat Mishra. 2021. Denial-of-service attack detection using machine learning in network-on-chip architectures. In *NOCS '21: International Symposium on Networks-on-Chip*, Tushar Krishna, John Kim, Sergi Abadal, and Joshua San Miguel (Eds.). ACM.

[64] Chamika Sudusinghe, Subodha Charles, and Prabhat Mishra. 2021. Network-on-chip attack detection using machine learning. In *Network-on-Chip Security and Privacy*. Springer.

[65] Adrian Tang, Simha Sethumadhavan, and Salvatore J. Stolfo. 2014. Unsupervised anomaly-based malware detection using hardware features. In *Research in Attacks, Intrusions and Defenses - 17th International Symposium, RAID (Lecture Notes in Computer Science)*, Angelos Stavrou, Herbert Bos, and Georgios Portokalidis (Eds.). Springer.

[66] Augusto Vega, Alper Buyuktosunoglu, and Pradip Bose. 2018. Towards "smarter" vehicles through cloud-backed swarm cognition. In *2018 IEEE Intelligent Vehicles Symposium, IV*. IEEE.

[67] Andrew J. Viterbi. 1967. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Trans. Inf. Theory* 13, 2 (1967), 260–269.

[68] Tianrui Wei, Nazerke Turtayeva, Marcelo Orenes-Vera, Omkar Lonkar, and Jonathan Balkind. 2023. Cohort: Software-oriented acceleration for heterogeneous SoCs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM.

[69] Young-Jin Yoon, Nicola Concer, Michele Petracca, and Luca P. Carloni. 2013. Virtual channels and multiple physical networks: Two alternatives to improve NoC performance. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 32, 12 (2013), 1906–1919.

[70] Florian Zaruba and Luca Benini. 2019. The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-GHz 64-Bit RISC-V core in 22-nm FDSOI technology. *IEEE Trans. Very Large Scale Integr. Syst.* 27, 11 (2019), 2629–2640.

[71] Iaroslav Zeigerman, Viktor Yershov, and Nikita Titov. 2019. m2cgen: Model 2 Code Generator. Retrieved from https://github.com/BayesWitnesses/m2cgen

[72] Joseph Zuckerman, Davide Giri, Jihye Kwon, Paolo Mantovani, and Luca P. Carloni. 2021. Cohmeleon: Learning-based orchestration of accelerator coherence in heterogeneous SoCs. In *MICRO '21: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. ACM.

Received 6 November 2024; revised 22 October 2025; accepted 26 October 2025